

Alex 和 Marvolo 不会出题了 - 1 - 番外

题意

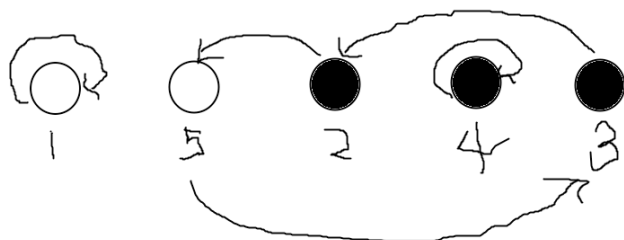
给出一排共 $n + m$ 个球，第 i 个球有权值 p_i ，前 n 个球颜色为黑，后 m 个球颜色为白，每次可以交换颜色不同的球，问最小交换多少次可以将所有球权值从小到大排序，即得到

$1, 2, \dots, n, n + 1, \dots, n + m$ 。

$1 \leq n + m \leq 200000$

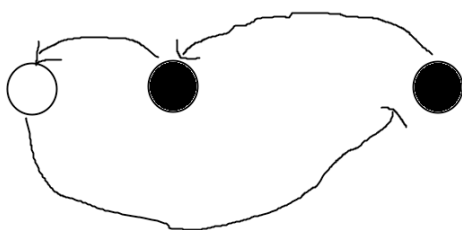
题解

第 i 个球权重为 p_i ，就代表着第 i 个球最终要到达第 p_i 个位置，以样例为例可以画出如下的图。



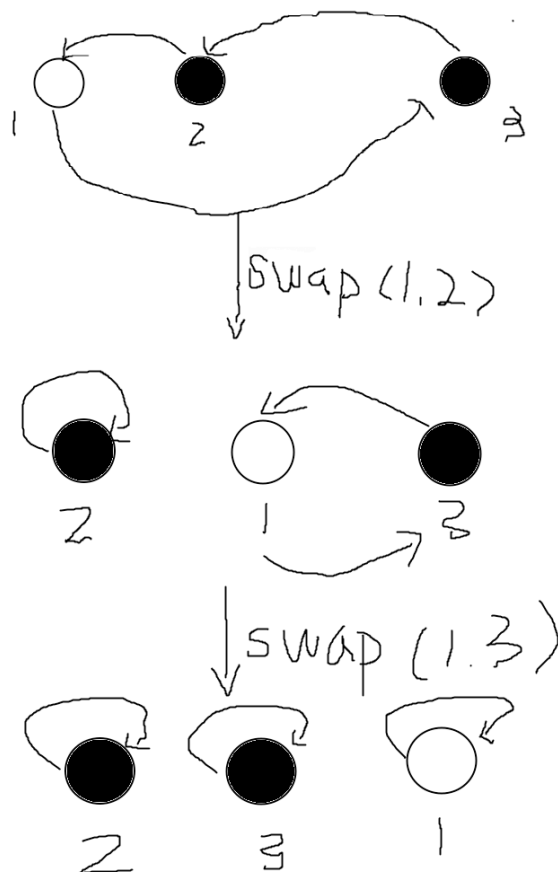
Paint X lite

对于已经满足 $i = p_i$ 的点可以暂时不用管了。先考虑形成环的部分，在这里就不用考虑标号了，如下图。



Paint X lite

显而易见这样一个环可以仅操作 2 次完成交换，过程如下图。



Paint X Lite

注意这里的 1, 2, 3 仅仅是为了表达方便，并非是权值。

由此我们可以想到一个简单的结论，即为环大小为 n 并且其中有黑有白，交换次数最小为 $n - 1$ 。

首先我们需要知道在没有黑白颜色的条件下，环大小为 n 的环交换次数最小已经是 $n - 1$ 了，证明如下。

首先 $n = 2$ 时显然交换一次就结束了， $f_2 = 1$ 。

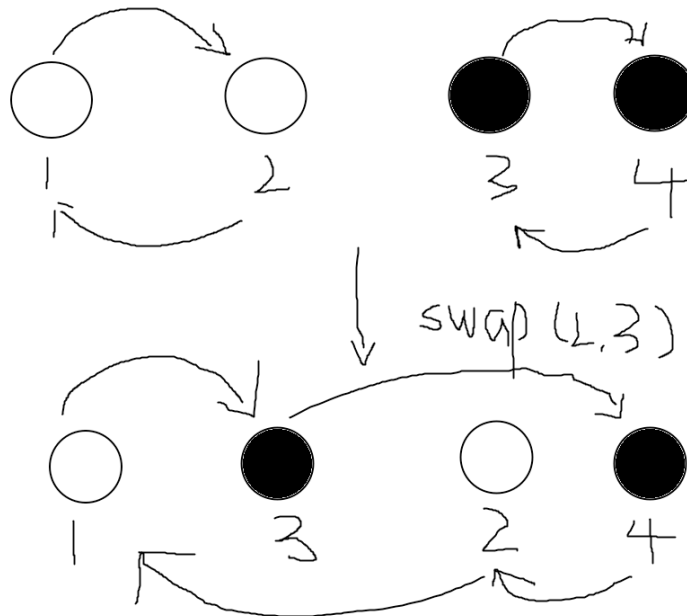
假设 $n < k, k \geq 3$ 成立， $n = k$ 时，由于不存在两个点交换后均到达对应的位置，因此每次操作只会最多减少环中的一个点或者将一个环拆成两个环。

对于减点的情况显然每次减一个点最优，在考虑因此

$f_k = 1 + \min(f_{k-1}, f_i + f_{k-i}), i \in [1, k-1]$ ，而 $\min$ 中的式子显然均为 $k - 2$ ，因此 $f_k = k - 1$ 。

在有黑有白的情况下，我们每次交换可以减少环中的一个黑或者一个白，在最后一次操作时减少一个黑与一个白，因此操作次数依旧为 $n - 1$ 。

现在已经解决了所有包含两种颜色的环，现在考虑环内颜色相同的情况。考虑现在有一个纯白环与一个纯黑环，我们需要怎么操作。



Paint X Lite

我们会发现对于两个不同颜色的纯色环，只需要交换一次就可以将其合并成一个包含两种颜色的大环。这样我们考虑有 a 个白环和 b 个黑环，由于黑白颜色是等价的，因此我们可以钦定 $a \leq b$ 。

显然我们连出来的环个数越多，所需要的交换次数越少，本质上交换次数为总点数减联通块个数，因此我们可以让前 $a - 1$ 个白环和 b 个黑环的其中 $a - 1$ 个黑环意义对应，剩下的 $b - a + 1$ 个黑环与最后一个白环连即可得到一个最优方案。

假设此时 $a = 0$ ，则外面一定有一个白点自己原本已经是答案了，否则就无解了，可以通过这个点和黑环交换得到两种颜色的环。

总体复杂度为 $O(n)$

代码

```
#include<bits/stdc++.h>
#define inf 0x3f3f3f3f
#define maxm 4020005
#define maxn 1000005
#define PII pair<int, int>
#define fi first
#define mkp make_pair
#define se second
#define ls (tot << 1)
#define rs (tot << 1 | 1)
typedef long long ll;
typedef unsigned long long ull;
using namespace std;
const ll mod = 1e9 + 7;
const double eps = 1e-6;
inline ll read(){
    ll x = 0, f = 1;char ch = getchar();
    while(ch > '9' || ch < '0'){if(ch == '-') f = -1;ch = getchar();}
```

```

        while(ch >= '0' && ch <= '9'){x = x * 10 + ch - '0';ch = getchar();}
        return x * f;
    }

    PII p[maxn];
    bool vis[maxn];
    int ans, sum;
    vector<int> H, B;

    int main(){
        int n = read(), m = read();
        for(int i = 1; i <= n; i++) p[i] = {read(), 1};
        for(int i = 1; i <= m; i++) p[i + n] = {read(), 0};
        int tot, cnt;
        for(int i = 1; i <= n + m; i++){
            if(vis[i]) continue;
            tot = 0, cnt = 0;
            for(int j = i; !vis[j]; j = p[j].fi){
                vis[j] = 1;
                tot++, cnt += p[j].se;
            }
            if(tot == 1) continue;
            int h = cnt, b = tot - cnt;
            if(h && b){ans += tot - 1;continue;}
            if(h) H.push_back(h);
            if(b) B.push_back(b);
            sum += h + b;
        }
        if(H.size() || B.size()){
            int mn = min(H.size(), B.size()) - 1, mx = max(H.size(), B.size());
            ans += mx;
            if(sum) ans += sum - mn - 1;
        }
        printf("%d\n", ans);
    }
}

```